IV Encontro do PGMAT
12 e 13 de março de 2026

Aleatoriedade vs Pseudoaleatoriedade: demonstrando a importância do gerador de chaves em Criptoprafia Simétrica através de uma simulação simplicada em *Python*

Apresentação de pôster

Aluna: Gisela Ceresér Kassick

Orientador: Dr. Daniel Roberto Cassar

Ilum Escola de Ciência

Centro Nacional de Pesquisa em Energia e Materiais

Campinas

Março, 2026

Introdução

A aleatoriedade é elemento central da criptografia moderna, indispensável à geração de chaves, vetores de inicialização, nonces e demais parâmetros críticos de segurança. A proteção de sistemas criptográficos depende da produção de informações que um adversário não consiga aprender ou prever; portanto, a qualidade da fonte aleatória constitui requisito essencial para a confidencialidade e a integridade das comunicações (GENNARO, 2006).

Do ponto de vista formal, uma sequência aleatória deve satisfazer três propriedades fundamentais: (i) distribuição uniforme; (ii) independência entre seus elementos; e (iii) impossibilidade de prever o restante da sequência a partir de qualquer subsequência observada (ZHANG et al., 2021). Esses critérios fornecem bases objetivas para a avaliação da qualidade estatística de sequências utilizadas em algoritmos criptográficos. A mensuração dessa qualidade pode ser realizada por diferentes métricas. A entropia quantifica o grau de incerteza associado a uma fonte (GENNARO, 2006), enquanto testes de correlação identificam dependências estruturais incompatíveis com a independência estatística (ZHANG et al., 2021).

Na criptografia simétrica, a geração da chave secreta é determinante: mesmo algoritmos matematicamente robustos tornam-se vulneráveis quando a chave provém de uma fonte previsível, reduzindo o espaço efetivo de busca e facilitando ataques de força bruta ou análise estatística (GENNARO, 2006). Nesse contexto, os geradores de números aleatórios desempenham papel central. Geradores verdadeiramente aleatórios (TRNGs) exploram fenômenos físicos imprevisíveis, como ruído térmico, para produzir sequências digitais (CUCU LAURENCIU; COTOFANA, 2018). Entretanto, processos físicos podem gerar saídas com vieses estatísticos; por isso, utilizam-se técnicas de extração baseadas em funções hash para suavizar a entropia e aproximar a distribuição do ideal uniforme (GENNARO, 2006). Já geradores pseudoaleatórios (PRNGs) expandem deterministicamente uma semente inicial. PRNGs convencionais — como o Mersenne Twister — não são adequados para fins criptográficos, pois seu estado interno pode ser reconstruído a partir de saídas observadas (VIGNA, 2019). Para aplicações sensíveis, empregam-se CSPRNGs, cujas saídas são indistinguíveis de sequências verdadeiramente aleatórias e permanecem imprevisíveis mesmo com conhecimento parcial do sistema (D'APRANO, 2015).

Diante desse referencial teórico, o presente trabalho tem como objetivo analisar e comparar a qualidade estatística e a adequação criptográfica de diferentes fontes de geração de números aleatórios, avaliando seu impacto na criptografia simétrica. Busca-se investigar em que medida distintas abordagens — físicas e algorítmicas — atendem aos critérios formais de aleatoriedade e preservam imprevisibilidade sob análise estatística.

A pergunta que orienta a pesquisa é: Diferenças na origem da aleatoriedade produzem variações estatísticas relevantes que impactam, de forma prática, a segurança de um sistema simples de criptografia simétrica? Espera-se que geradores não projetados para fins criptográficos apresentem vulnerabilidades exploráveis, mesmo quando exibem propriedades estatísticas aparentemente adequadas. Em contraste, geradores criptograficamente seguros e fontes físicas devidamente tratadas tendem a apresentar desempenho estatístico semelhante e resistência prática equivalente em aplicações de criptografia simétrica.

Metodologia

O presente trabalho adota uma abordagem experimental e comparativa, combinando análise estatística da aleatoriedade com avaliação prática de segurança em criptografia simétrica. Para a obtenção e processamento dos dados experimentais, foi desenvolvido um código em Python integrado a um Arduino por meio da biblioteca *serial*, possibilitando a coleta de sequências binárias provenientes de fonte física e sua posterior análise e aplicação criptográfica.

1. Geração das sequências

Foram consideradas quatro fontes distintas de geração de sequências binárias, com 256 bits por experimento:

1. **Gerador pseudoaleatório padrão (random)** – baseado no algoritmo Mersenne Twister, adequado para simulações numéricas, mas não projetado para segurança criptográfica (Python Software Foundation, 2026).
2. **Gerador criptograficamente seguro (secrets)** – módulo recomendado para geração de dados sensíveis, fundamentado em fontes seguras do sistema operacional (Python Software Foundation, 2026).
3. **Fonte física bruta** – coleta de bits provenientes de ruído físico captado por microcontrolador via comunicação serial. O ruído térmico constitui fonte física de imprevisibilidade explorada em geradores de números aleatórios (Cucu Laurenciu; Cotofana, 2018).
4. **Fonte física com extração por hash** – aplicação da função SHA-256 sobre os bits físicos coletados, utilizando a biblioteca hashlib (Python Software Foundation, 2026), com o objetivo de reduzir vieses estatísticos e concentrar a entropia.

2. Coleta da fonte física

A sequência física foi obtida por meio de um Arduino Leonardo configurado, em linguagem C++, para realizar leituras em pino analógico do ruído térmico de Johnson–Nyquist de um resistor de 10MΩ, amplificado por um Amplificador Operacional LM358 (CUCU LAURENCIU; COTOFANA, 2018; WALTER et al, 2023). A configuração do circuito é apresentada na Figura 1:

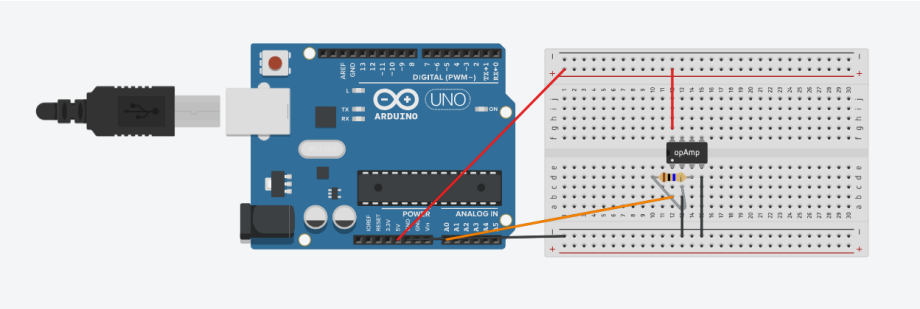

Figura 1. Sistema físico simulado no Tinkercad para ilustrar a conexão feita em um Arduino Leonardo físico para a execução do experimento. **FONTE**: Autor (2026).

O programa realiza leituras sucessivas e extrai o bit menos significativo de cada valor, transmitindo-o via comunicação serial. Os bits recebidos pelo código em Python compõem a sequência utilizada como fonte física bruta no experimento.

3. Análise estatística

Cada sequência foi submetida a três métricas principais: a Entropia de Shannon, para estimar o grau de incerteza da distribuição binária; a autocorrelação, para detectar dependência linear entre bits consecutivos; e o teste qui-quadrado, para avaliar compatibilidade com distribuição uniforme (50% zeros e 50% uns).

Essas métricas permitem comparar as fontes quanto à uniformidade, independência e imprevisibilidade estatística.

4. Derivação de chave e configuração criptográfica

Os 256 bits gerados foram convertidos em bytes e utilizados para derivar a chave AES-256, enquanto os primeiros 128 bits da sequência foram utilizados como vetor de inicialização (IV).

A cifra empregada foi o Advanced Encryption Standard (AES) em modo CTR (Counter Mode), implementado por meio da biblioteca cryptography (PyCA Cryptography, 2026). O AES foi escolhido por ser um padrão consolidado de criptografia simétrica, amplamente utilizado e com suporte a chaves de 256 bits, adequado à análise da qualidade do material criptográfico (BLAZHEVSKI et al., 2012). O modo CTR foi adotado por operar a cifra de bloco como um gerador de fluxo baseado em contador, permitindo processamento eficiente e dependência direta da chave e do vetor de inicialização (BLAZHEVSKI et al., 2012).

5. Simulação de ataques

Para avaliar a segurança prática das chaves derivadas, foram implementadas duas estratégias experimentais:

I. Reconstrução de seed – exploração da natureza determinística do gerador random, por meio da testagem sistemática de possíveis sementes até a recuperação da mensagem original.
II. Força bruta limitada – geração repetida de chaves candidatas para tentativa de descriptografia bem-sucedida, simulando um cenário de adivinhação probabilística.

O critério de sucesso foi a recuperação integral da mensagem original, bem como o registro do tempo necessário para cada ataque.

A inclusão do ataque de reconstrução de *seed* baseia-se no modelo de segurança da criptografia moderna, segundo o qual o adversário conhece o algoritmo e o método de geração de chaves, sendo secreta apenas a chave específica (KATZ; LINDELL, 2007). Assim, a segurança não pode depender do sigilo do procedimento. Se o gerador for determinístico e não criptograficamente seguro, sua previsibilidade pode ser explorada para reduzir o esforço de quebra (GENNARO, 2006). A reconstrução da *seed*, portanto, simula um atacante que conhece o mecanismo de geração e busca recuperar seu estado inicial.

6. Análise comparativa

A avaliação final integra resultados estatísticos (entropia, autocorrelação e uniformidade) e desempenho sob ataque, permitindo analisar se diferenças na origem da aleatoriedade se refletem em variações concretas na segurança de um sistema criptográfico baseado em AES-CTR.

Resultados

A partir dos dados obtidos em 10.000 iterações independentes, foram construídos os gráficos comparativos apresentados na Figura 2, que sintetizam a distribuição das métricas estatísticas analisadas ao longo das execuções.

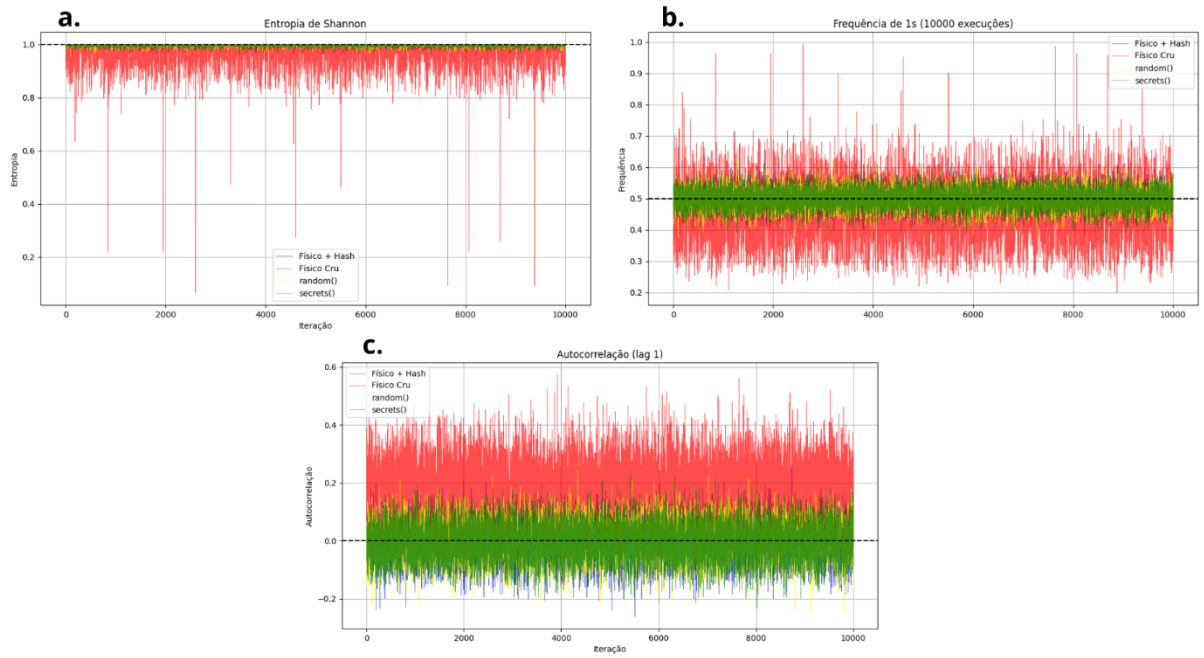

Figura 2. a) Gráfico da entropia de Shannon de cada fonte de acordo com a iteração. b) Gráfico da frequência de 1s de cada fonte de acordo com a iteração. c) Gráfico da autocorrelação dos dados de cada fonte de acordo com a iteração. **FONTE:** Autor (2026).

No caso do gerador físico bruto, observaram-se diferenças estatisticamente relevantes em relação aos geradores pseudoaleatórios. A frequência média de bits 1 foi de aproximadamente 0,466, indicando um viés em relação ao valor ideal de 0,5. A entropia média também foi menor (≈ 0,972 bits por bit), refletindo essa assimetria na distribuição dos bits. O resultado mais significativo foi a autocorrelação média positiva (≈ 0,191), evidenciando dependência temporal entre bits consecutivos. Esse comportamento é esperado em sistemas experimentais reais, pois fontes físicas de entropia apresentam imperfeições decorrentes de limitações do circuito e de fatores ambientais,

como temperatura, variações na tensão de alimentação e envelhecimento de componentes (CUCU; COTOFANA, 2018).

Além disso, devido às características do amplificador operacional utilizado, parte da variação medida pelo Arduino pode estar associada à tensão de offset de entrada, cuja magnitude pode ser comparável à do próprio ruído térmico. Esse efeito pode introduzir assimetria na distribuição dos valores digitalizados pelo ADC, contribuindo para o viés observado e para a correlação entre amostras consecutivas. Embora essa fonte física não tenha sido decifrada nos ataques realizados, a presença de viés estatístico e autocorrelação indica que ela poderia, em princípio, tornar-se vulnerável a ataques mais sofisticados que explorem essas dependências estruturais.

Uma abordagem comum para mitigar imperfeições de fontes físicas é a aplicação de funções hash criptográficas como extratores de entropia (GENNARO, 2006). Após a aplicação de SHA-256 sobre os dados físicos, observou-se redução significativa da autocorrelação, que passou de aproximadamente 0,191 para −0,005, valor comparável ao dos geradores pseudoaleatórios. A frequência média de bits também passou a ≈ 0,500, e a entropia média aumentou para ≈ 0,997 bits por bit, próxima do máximo teórico. Além disso, os p-valores médios dos testes de uniformidade ficaram próximos de 0,5, indicando que a distribuição dos bits gerados se aproxima de uma distribuição uniforme. Nos testes de ataque realizados, a sequência resultante também não apresentou vulnerabilidades detectáveis, reforçando o papel do hash como etapa fundamental na construção de geradores seguros.

O gerador random() apresentou desempenho estatístico próximo do ideal. A frequência média de bits 1 foi ≈ 0,4997, a entropia média ≈ 0,997 bits por bit, e a autocorrelação média ≈ −0,004, valores muito próximos dos esperados para uma sequência aleatória. Os p-valores médios (≈ 0,504) também indicaram compatibilidade com uma distribuição uniforme dos bits. Apesar dessas propriedades estatísticas favoráveis, esse gerador foi o único comprometido nos ataques realizados. A partir da observação de saídas suficientes, foi possível reconstruir seu estado interno e prever valores futuros da sequência. Isso ocorre porque o algoritmo utilizado, o Mersenne Twister, é um gerador pseudoaleatório determinístico baseado em transformações lineares, o que permite a recuperação do estado interno em determinadas condições. Assim, embora apresente excelente desempenho estatístico para simulações, ele não foi projetado para aplicações criptográficas (VIGNA, 2025).

Por outro lado, o gerador secrets() apresentou métricas estatísticas semelhantes às observadas em random(). A frequência média de bits foi ≈ 0,5001, a entropia média ≈ 0,997 bits por bit, e a autocorrelação média ≈ −0,004, com p-valores médios próximos de 0,5, novamente indicando compatibilidade com uma distribuição uniforme dos bits. A diferença fundamental é que secrets() foi projetado para aplicações criptográficas, utilizando fontes de entropia do sistema operacional combinadas com mecanismos de geração seguros. Nos testes de ataque realizados, o gerador não apresentou vulnerabilidades detectáveis, mantendo imprevisibilidade mesmo sob observação das sequências geradas, em concordância com sua recomendação oficial para uso criptográfico (PYTHON SOFTWARE FOUNDATION, 2024).

Conclusão

Em síntese, os resultados obtidos reforçam um princípio fundamental da criptografia: boas propriedades estatísticas não são suficientes para garantir segurança criptográfica. O gerador random() apresentou métricas estatísticas praticamente ideais, mas permaneceu vulnerável a ataques devido à sua natureza determinística. Por outro lado, a fonte física bruta apresentou imperfeições experimentais, como viés e autocorrelação, que podem reduzir a entropia efetiva da sequência gerada. A aplicação de funções hash criptográficas demonstrou ser um mecanismo eficaz para mitigar essas imperfeições e produzir sequências mais adequadas para aplicações de segurança. Dessa forma, os resultados evidenciam a importância de utilizar algoritmos e arquiteturas especificamente projetados para criptografia ao construir sistemas de geração de números aleatórios destinados a aplicações seguras.

Referências

BLAZHEVSKI, D.; et al. Modes of Operation of the AES Algorithm. 2012.

CUCU LAURENCIU, N.; COTOFANA, S. D. Low Cost and Energy, Thermal Noise Driven, Probability Modulated Random Number Generator. Delft University of Technology, 2018.

D'APRANO, S**.** PEP 506 – Adding a Secrets Module to the Standard Library. 2015. Disponível em: https://peps.python.org/pep-0506/. Acesso em: 26 fev. 2026.

GENNARO, R. Randomness in Cryptography. *IEEE Security & Privacy*, v. 4, n. 2, p. 64-67, 2006.

KATZ, J.; LINDELL, Y. Introduction to Modern Cryptography. 2007.

MENGDI, Z. et al. Overview of Randomness Test on Cryptographic Algorithms. *Journal of Physics: Conference Series*, v. 1861, n. 1, 012009, 2021.

PYTHON SOFTWARE FOUNDATION. random — geração de números pseudoaleatórios. Disponível em: https://docs.python.org/3/library/random.html. Acesso em: 26 fev. 2026.

PYTHON SOFTWARE FOUNDATION. secrets — geração de números seguros para gerenciamento de dados sensíveis. Disponível em: https://docs.python.org/3/library/secrets.html. Acesso em: 26 fev. 2026.

PYTHON SOFTWARE FOUNDATION. hashlib — algoritmos de hash seguros. Disponível em: https://docs.python.org/3/library/hashlib.html. Acesso em: 26 fev. 2026.

PYCA CRYPTOGRAPHY. Cryptography Documentation. Disponível em: https://cryptography.io. Acesso em: 26 fev. 2026.

VIGNA, S. *It Is High Time We Let Go of the Mersenne Twister*. arXiv:1910.06437, 2019. Disponível em: https://arxiv.org/abs/1910.06437. Acesso em: 26 fev. 2026

WALTER, Daniela; BÜLAU, André; ZIMMERMANN, André. Review on excess noise measurements of resistors**.** *Sensors*, v. 23, n. 3, p. 1107, 2023. DOI: https://doi.org/10.3390/s23031107